SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA10 – SOFTWARE ENGINEERING

ASSIGNMENT 3 – CO3

Full-Stack Development and Containerization

of the Smart Campus Facility Platform

CO3: Develop full-stack applications using version control systems like Git/GitHub and containerization tools like Docker and Kubernetes.

Student Name	ATHIN SIVA.S	**Register Number**	192524237
Department / Section	AIDS	**Academic Year / Semester**	2025
Faculty Name		**Nature of Assignment**	**Individual**
Duration	**1 Week**	**Total Marks**	**50**
Date of Submission	5/11/2026	**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Define Application Scope and Select Technology Stack	5		
2	Create Integrated Application Design	8		
3	Develop Working Full-Stack Application	10		
4	Establish and Demonstrate Version-Control Workflow	7		
5	Create and Validate Reproducible Container Deployment	8		
6	Adapt Application for Concurrent Access and Deployment Changes	7		
7	Individual Technical Reflection	5		
	TOTAL	50		

OVERALL REMARKS

Student Signature	Faculty Signature	Program Director

MANDATORY SUBMISSION EVIDENCE CHECK

Task	Evidence Expected	Status	Note
1	Stakeholders, workflows, scope boundaries, user stories, acceptance criteria, technology comparison and measurable success criteria.		
2	Integrated component architecture, data model, API contracts, state models, ownership rules and consistency constraints.		
3	Full-stack application screens and workflows, backend integration, role-based permissions, explainable triage and end-to-end demonstration evidence.		
4	Repository structure, branch strategy, commits, pull requests or reviews, conflict resolution and recovery workflow.		
5	Dockerfile and Compose design, services and dependencies, environment configuration, persistent storage, deployment commands and reproducibility validation.		
6	Impact analysis for concurrent users and deployment changes, revised architecture/configuration, consistency and synchronization rules, bounded validation and trade-off analysis.		
7	Reflection on a significant technical decision, evidence from the artefacts, limitations, unresolved risk and next implementation step.		

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All tasks (Q1–Q7) answered with the required evidence	☐ Yes ☐ Partly ☐ No
Application stakeholders, scope, user stories and technology stack documented (Q1)	☐ Yes ☐ Partly ☐ No
Integrated architecture, data model, API contracts and state models included (Q2)	☐ Yes ☐ Partly ☐ No
Full-stack application workflow and role-based behaviour demonstrated (Q3)	☐ Yes ☐ Partly ☐ No
Git/GitHub version-control workflow and conflict handling documented (Q4)	☐ Yes ☐ Partly ☐ No
Docker/container deployment configuration and reproducibility validation included (Q5)	☐ Yes ☐ Partly ☐ No
Concurrent access, deployment changes and consistency risks addressed (Q6)	☐ Yes ☐ Partly ☐ No
Individual reflection explains decisions, limitations and next steps (Q7)	☐ Yes ☐ Partly ☐ No
Diagrams, tables, API examples and interface evidence are labelled clearly	☐ Yes ☐ Partly ☐ No
Illustrative or simulated evidence is distinguished from actual execution results	☐ Yes ☐ Partly ☐ No
No unfilled template placeholders; design, implementation and validation are consistent	☐ Yes ☐ Partly ☐ No

Viva readiness: able to explain architecture, Git workflow and container deployment	☐ Yes ☐ No

Report Overview

This report presents a representative full-stack Smart Campus Facility Platform. It combines a Vue.js frontend, Django REST API and PostgreSQL database. The first release supports facility discovery, booking, maintenance-ticket intake, explainable triage, technician assignment, inspection, approval and closure. Git provides traceable source history, while Docker Compose provides a reproducible multi-container deployment.

Evidence note: the diagrams and interface/terminal-style visuals in this report are illustrative design and demonstration aids generated for this assignment. They are not screenshots captured from a live student repository, running application, Docker host or GitHub account. Replace them with genuine screenshots and measured results after executing the implementation. Proposed code/configuration examples are not claims of completed deployment.

Problem Statement

A university requires an integrated campus operations release that supports facility booking or sensor-alert intake, explainable maintenance triage, technician assignment, inspection, approval and verified completion. Separate frontend, backend and database changes create integration problems, while inconsistent developer environments make deployment difficult to reproduce. The system must preserve persistent records, server-enforced permissions and valid booking and ticket transitions. It must also handle duplicate requests, overlapping reservations, conflicting updates and two backend instances.

Q1 – Define the Application Scope and Select the Technology Stack (5 Marks)

1.1 Stakeholders and workflows

Stakeholder	Main need	Typical action
Student / staff	Find and reserve a facility	Search availability, request slot, view status

Facility manager	Control restricted spaces and capacity	Approve or reject restricted booking
Technician	Work from an accurate queue	Accept ticket, inspect, add notes, resolve
Campus administrator	Manage users, facilities and policies	Configure roles, campus and SLA rules
Helpdesk operator	Track maintenance service	Create, classify and assign tickets
Management	Understand utilization and service quality	Review booking, ticket and SLA dashboard

1.2 Scope boundaries

Included in the first release: facility catalogue, availability calendar, booking request and approval, ticket creation from a controlled sensor event or manual intake, deterministic triage suggestions, technician assignment, inspection notes, resolution, role-based closure, audit history and a basic dashboard. Excluded: payment processing, predictive occupancy AI, biometric access, building automation control, live IoT fleet management, advanced mobile apps and multi-cloud orchestration.

1.3 User stories and acceptance criteria

ID / user story	Measurable acceptance criteria
US01 As a student, I can browse facilities.	Catalogue returns active facilities with capacity and location; inactive facilities cannot be booked.
US02 As a user, I can request a time slot.	Valid request is saved with Requested state and unique booking ID.
US03 As a manager, I can approve restricted bookings.	Only FacilityManager role can approve; decision and actor are audited.
US04 As a user, I can view booking status.	Owner can view own booking; another user receives 403/404.
US05 As helpdesk, I can create a maintenance ticket.	Same sensor event ID creates only one ticket.
US06 As a technician, I can resolve assigned work.	Only assigned technician or authorized supervisor can add resolution and close.

1.4 Technology comparison

Layer	Option A (selected)	Option B	Reason
Frontend	Vue.js	React	Vue has a compact component model and is suitable for a small student project.

Backend	Django REST Framework	Node.js + Express	Django provides authentication, ORM, serializers and admin support.
Database	PostgreSQL	MongoDB	Relational constraints and transactions suit bookings and audit records.
Version control	Git + GitHub	Git + GitLab	GitHub supports branches, pull requests, issues and release tags.
Deployment	Docker Compose	Kubernetes	Compose is simpler and lower-cost for the bounded first release.

1.5 Success criteria

- A valid booking is stored and can be retrieved after API restart.
- Overlapping active reservations for the same facility and time are rejected by the backend/database.
- Duplicate sensor event IDs do not create duplicate tickets.
- Unauthorized role actions return a clear 403 response and do not change state.
- A clean deployment can start from documented commands and retain database records using a named volume.
- The repository contains at least two feature branches, reviewed change evidence and a stable release tag.

Q2 – Create the Integrated Application Design (7 Marks)

2.1 Component architecture

The browser communicates with the Vue interface. The interface calls Django REST endpoints over HTTPS/JSON. Django authenticates users, validates input, checks roles and applies business rules. PostgreSQL is the authoritative store for users, facilities, bookings, tickets, approvals and audit events. The frontend provides guidance but is never trusted to enforce authorization or consistency.

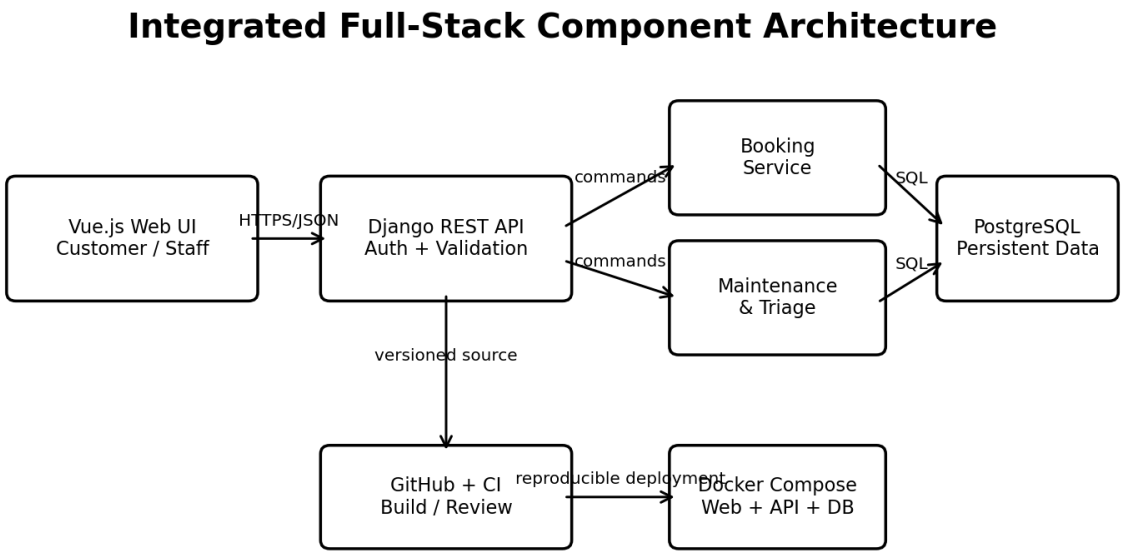


Figure 1. Illustrative component architecture for the full-stack platform.

2.2 Data model

Entity	Important fields	Relationship / rule
User	id, username, role, campus_id	Owns bookings; role controls permitted operations.
Facility	id, name, capacity, restricted, campus_id	Has many bookings; capacity and active flag validated.
Booking	id, user_id, facility_id, start_at, end_at, state, version	Facility/time interval reservation; version guards updates.
MaintenanceTicket	id, event_id, category, priority, assignee_id, state, version	Unique event_id prevents duplicate sensor tickets.
SLAPolicy	id, category, response_minutes, resolution_minutes	Defines measurable service targets.
Approval	id, object_type, object_id, reviewer_id, decision, decided_at	Restricted action requires authorized reviewer.
AuditEvent	id, actor_id, action, object_id, old_state, new_state, timestamp	Append-only record of critical state changes.

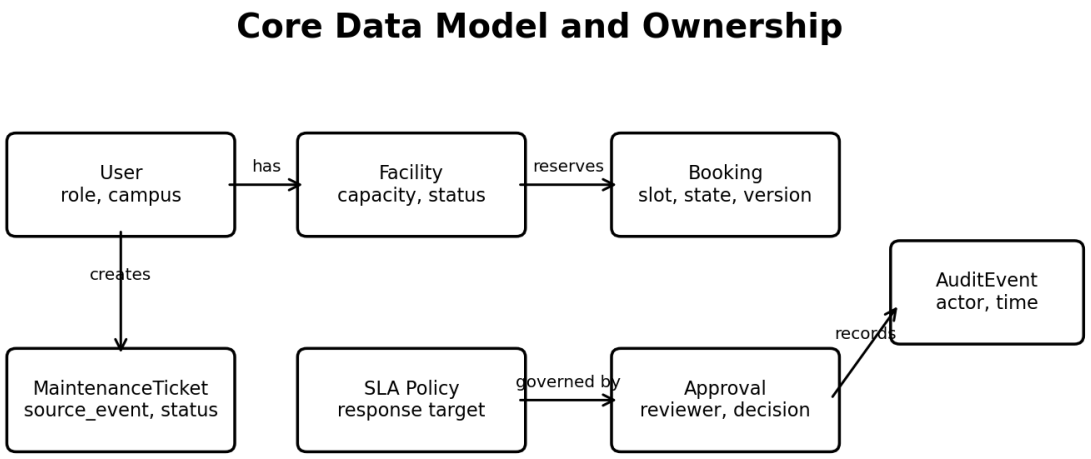


Figure 2. Illustrative core data model and relationships.

2.3 Principal API contracts

Method and endpoint	Purpose	Success / failure
GET /api/facilities/	List active facilities	200 with facility records
GET /api/facilities/{id}/availability/	Check availability for a time range	200 slots; 400 invalid range
POST /api/bookings/	Create booking request	201 created; 409 overlap; 400 invalid data
POST /api/bookings/{id}/decision/	Approve or reject restricted booking	200 decision; 403 wrong role; 409 stale state
POST /api/tickets/	Create manual or sensor ticket	201 created; duplicate event returns existing ticket
POST /api/tickets/{id}/assign/	Assign technician	200 assigned; 403 unauthorized
POST /api/tickets/{id}/resolve/	Record inspection and resolution	200 resolved; 409 invalid transition
POST /api/tickets/{id}/close/	Close verified ticket	200 closed; 403/409 on prohibited action

2.4 State models and consistency rules

Booking states are Requested, Approved, Confirmed, In Use, Completed, Cancelled and Rejected. A restricted booking must pass manager approval before confirmation. A booking cannot move from Cancelled to In Use. Ticket states are New, Triaged, Assigned, Inspecting, Pending Parts, Resolved

and Closed. Closure requires a resolution note and an authorized actor. Every transition records actor, timestamp and previous/new state.

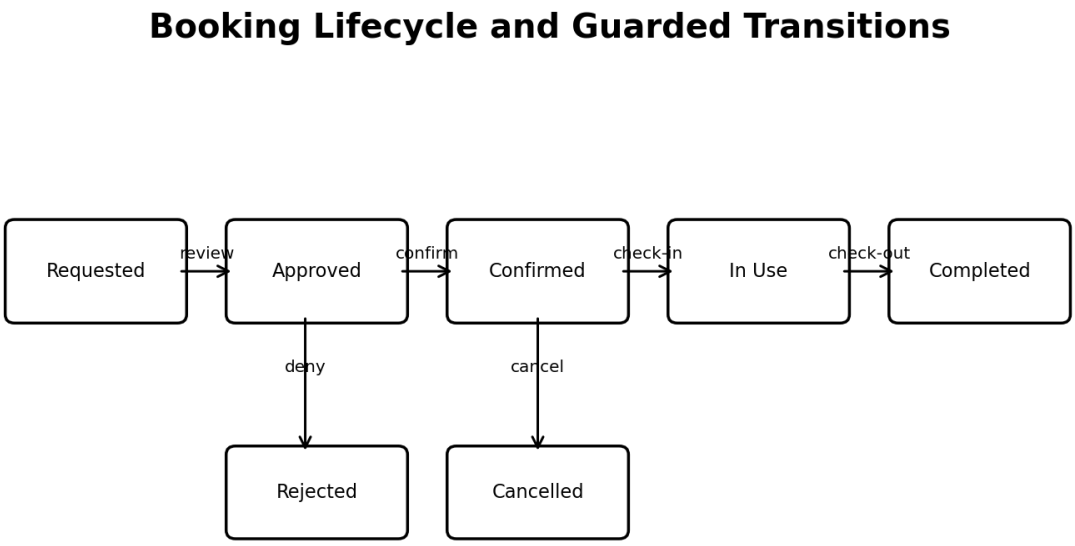


Figure 3. Illustrative booking state model; ticket transitions follow the guarded rules described above.

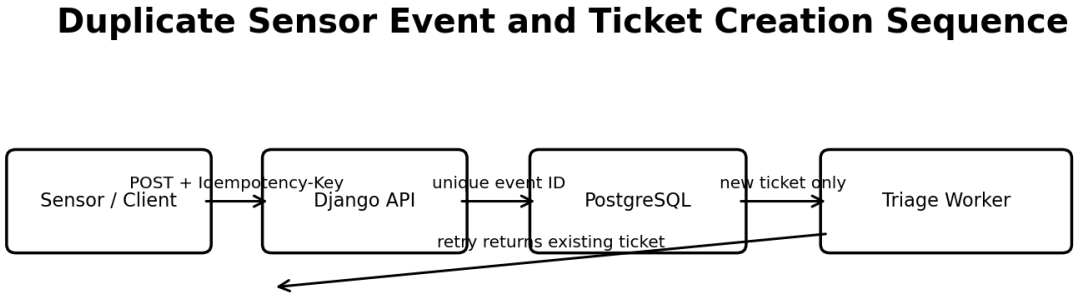


Figure 4. Illustrative idempotent sensor-event intake sequence.

Double booking is prevented by checking overlapping intervals on the server inside a transaction and enforcing a database exclusion constraint where supported. If the selected database design cannot use

a range exclusion constraint, use a serialized transaction/lock around the facility and time range, plus a final conflict query. Duplicate ticket creation is prevented with a unique constraint on source event identity. Conflicting ticket updates use an expected version number: update succeeds only when the submitted version matches the current version.

Validation responsibility is layered: Vue validates required fields for usability; Django repeats all security and business validation; PostgreSQL enforces uniqueness, foreign keys, non-null fields and transactional consistency. The database is authoritative.

Q3 – Develop a Working Full-Stack Application (12 Marks)

3.1 Main application screens

The dashboard summarizes facility availability, active bookings, open maintenance tickets and SLA risks. The booking view allows users to choose a facility and time interval. The maintenance queue displays category, suggested priority, assignee and current state. The ticket detail view supports inspection notes, pending-work reasons, resolution and authorized closure.

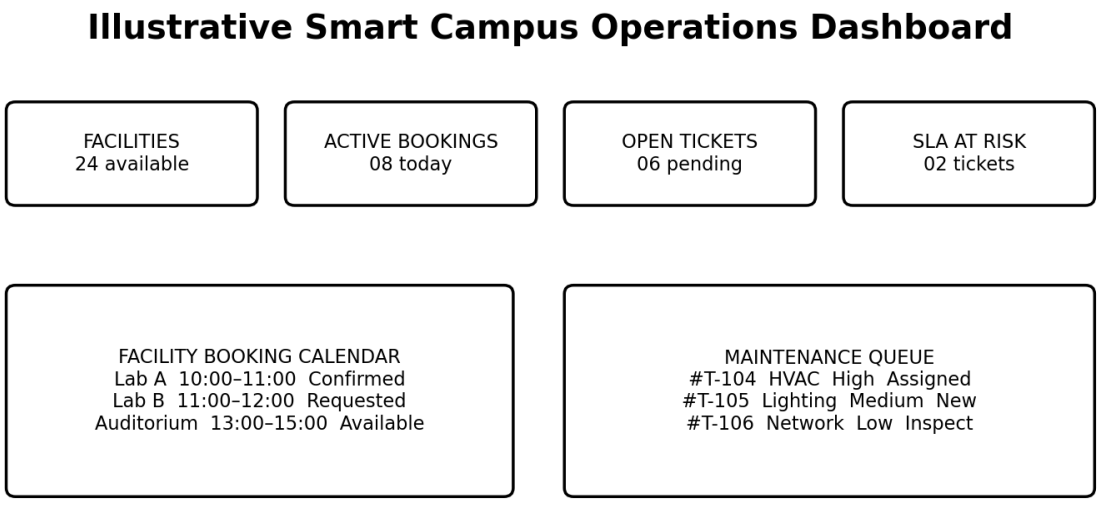


Figure 5. Illustrative dashboard mock-up. Replace with an actual browser screenshot after running the application.

3.2 Explainable triage

For the representative release, triage is deterministic rather than trained machine learning. Example rules: an HVAC temperature alert above the configured critical threshold is categorized as HVAC and assigned High priority; a repeated low-severity lighting alert is Medium; a non-critical cosmetic issue is Low. The ticket stores the matched rule and input evidence so a technician can understand why the suggestion was made. A human operator may correct the category or priority, with the override reason recorded.

Input condition (illustrative)	Suggested category	Priority	Reason displayed
Temperature exceeds critical threshold	HVAC	High	Critical temperature threshold exceeded.
Power interruption detected	Electrical	High	Safety/service interruption rule matched.
Repeated lighting fault	Lighting	Medium	Repeated fault count reached configured threshold.
Cosmetic damage report	General	Low	No safety or service-impact rule matched.

3.3 Role restrictions

Role	Allowed actions	Restricted actions
Student / staff	Browse facilities, request booking, view own records	Approve bookings, assign tickets, close work
Facility manager	Approve/reject restricted booking, manage facility availability	Change system administrator roles
Helpdesk operator	Create tickets, classify, assign	Approve own restricted booking or bypass closure checks
Technician	View assigned tickets, inspect, resolve	Close ticket without required resolution evidence
Administrator	Manage users, roles, campuses and policies	Erase audit history through normal application UI

3.4 End-to-end demonstration plan

- Sign in as a staff user and select an available facility.
- Submit a booking request; verify that the record appears in the database.
- Sign in as a facility manager and approve a restricted booking.

- Generate a controlled sensor event with a unique event ID; verify one ticket is created.
- Assign the ticket to a technician, add inspection notes and resolve it.
- Close the ticket using an authorized role and verify the audit trail.
- Open the linked GitHub issue or delivery record and show its disclosed integration mode.

The above is a reproducible demonstration procedure, not a claim that the live application was executed in this document. Capture actual browser and database evidence when the implementation is run.

Q4 – Establish and Demonstrate a Version-Control Workflow (6 Marks)

4.1 Repository structure

A suitable repository structure separates frontend, backend, deployment configuration, documentation and tests. Generated dependencies, virtual environments, local database files, secrets and build output must be excluded.

Path	Purpose
frontend/	Vue source, components, views and package manifest
backend/	Django project, apps, serializers, permissions and migrations
deploy/	Dockerfile and Compose configuration
tests/	API, permission, overlap, idempotency and state-transition tests
docs/	Architecture, API contracts, setup and evidence notes
.env.example	Names of required variables with safe sample values; no secrets
.gitignore	Excludes .env, venv, node_modules, __pycache__, *.pyc and local data

4.2 Branch and review workflow

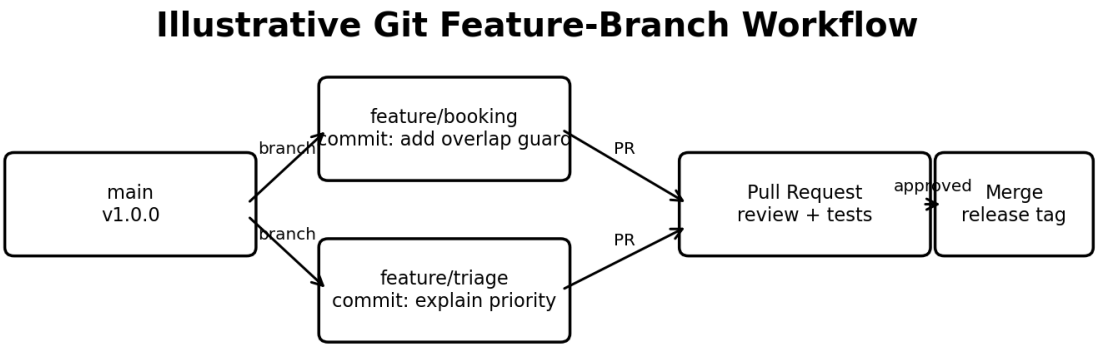


Figure 6. Illustrative feature-branch and pull-request workflow.

Step	Example command / action	Purpose
Create branch	git switch -c feature/booking-overlap	Isolate booking change.
Commit	git add . && git commit -m "Add booking overlap validation"	Record a meaningful unit of work.
Second branch	git switch -c feature/ticket-triage	Separate triage implementation.
Push	git push -u origin feature/ticket-triage	Publish branch for review.
Review	Open a GitHub pull request; inspect diff and tests	Request review and document feedback.
Merge	Merge only after checks pass	Integrate reviewed change.
Release	git tag -a v1.0.0 -m "First campus release"	Identify reproducible source version.

4.3 Conflict resolution

A deliberate conflict can be introduced by editing the same README setup line differently on two branches. Merge one branch into the other, inspect conflict markers, choose the correct combined instructions, run tests, then commit the resolution. For an individual assignment, describe this as a

simulated second contribution and self-review; do not claim another person reviewed it unless that actually happened.

4.4 Recovery

A stable tag identifies the known release. If a later change is unsuitable, revert the offending commit rather than rewriting shared history. Branches and commits make it possible to compare changes and restore a previously working state.

Q5 – Create and Validate a Reproducible Container Deployment (8 Marks)

5.1 Deployment design

Docker Compose is appropriate for the first release because the application has three principal services and a modest operating scope. The web container serves the Vue build, the API container runs Django/Gunicorn, and PostgreSQL stores persistent records. A named volume preserves database files when the database container is recreated. Secrets are supplied through environment variables or a local untracked .env file; production secrets should use a managed secret store.

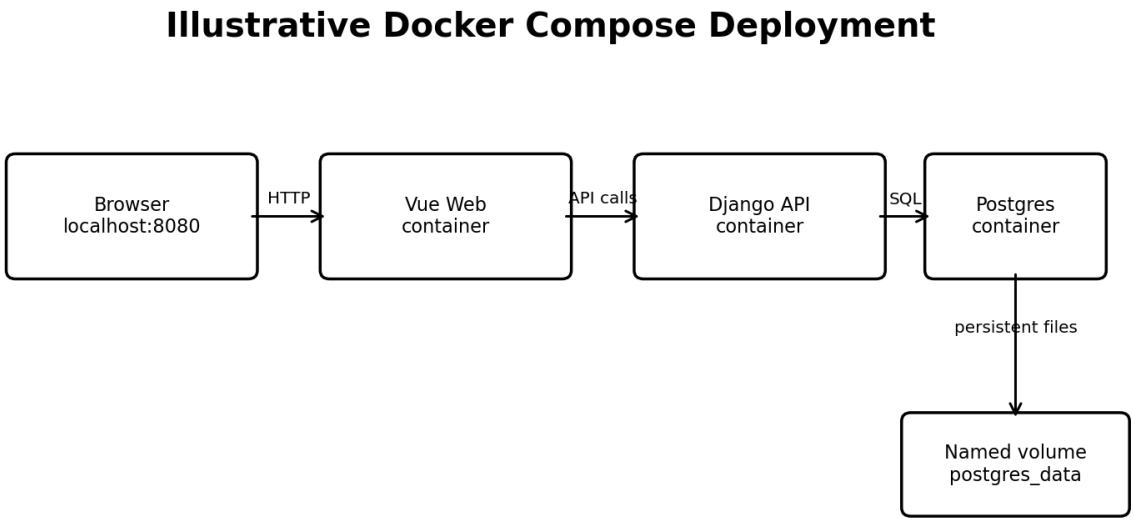


Figure 7. Illustrative Docker Compose service layout.

5.2 Example Compose configuration

```
services:
  web:
    build: ./frontend
    ports: ["8080:80"]
    depends_on: [api]
  api:
    build: ./backend
    command: gunicorn config.wsgi:application --bind 0.0.0.0:8000
    environment:
      DATABASE_URL: ${DATABASE_URL}
      DJANGO_SECRET_KEY: ${DJANGO_SECRET_KEY}
    depends_on: [db]
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: campus
      POSTGRES_USER: campus_app
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
volumes:
  postgres_data:
```

5.3 Deployment commands

```
cp .env.example .env
# Edit .env with local development values
docker compose build
docker compose up -d
docker compose exec api python manage.py migrate
docker compose exec api python manage.py createsuperuser
docker compose ps
docker compose logs -f api
docker compose down
# Stop containers but preserve named volume:
docker compose down
# Destructive reset only when intentionally required:
docker compose down -v
```

5.4 Persistence verification

Create a booking, record its identifier, restart only the API container, then retrieve the same booking through the API. Next restart the database container without deleting the named volume and verify the record again. A named volume is persistence for container lifecycle, not a backup. A separate backup and restore test is required for disaster recovery.

5.5 Clean-environment reproducibility

A clean test should clone the tagged repository on a fresh machine, create configuration from .env.example, build images, apply migrations and run the booking/ticket journey. Record OS, Docker version, commit/tag, commands, exit status and test results. Do not state that the deployment is reproducible until this procedure has been executed.

Q6 – Adapt the Application for Concurrent Access and Deployment Changes (8 Marks)

6.1 Impact analysis

The changed requirement expands from one environment to six campuses, with a planned peak of 300 users and two API instances sharing the database. The main risks are overlapping bookings, stale ticket updates, duplicate sensor/developer requests, and instance restart during a request. The API must remain stateless between requests; authoritative booking, ticket and audit state must live in shared persistence.

6.2 Revised deployment and consistency

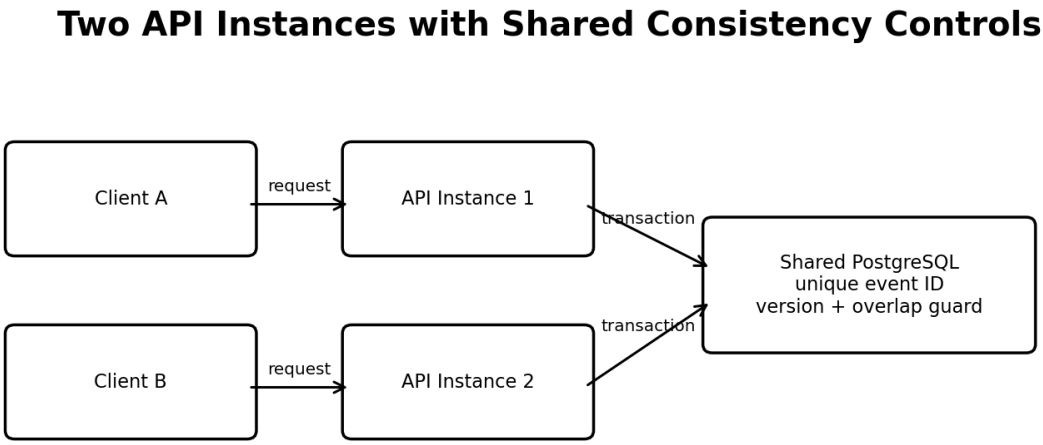


Figure 8. Two API instances share one authoritative database and common consistency rules.

Both API instances use the same database schema, authentication configuration and policy version. Booking overlap checks run transactionally and are enforced at database level where possible. Ticket updates include an expected version: UPDATE ... WHERE id = ? AND version = expected_version; a successful update increments version. If no row is updated, the API returns HTTP 409 Conflict and asks the user to reload. Sensor events and external issue creation use stable idempotency keys with a unique database constraint. After a timeout, the client retries with the same key and receives the prior result rather than creating a second effect.

6.3 Additional validation scenarios

Scenario	Expected behaviour	Evidence to collect
Two users reserve same lab/time	Only one active booking commits; other receives 409.	Parallel API test; database rows and responses.
Two technicians update same ticket version	First update succeeds; stale second update receives 409.	Two requests using same version; audit entries.
Sensor event retried after timeout	One ticket exists for event ID; retry returns same ticket.	Repeat POST with same idempotency key.
API instance 1 restarts	Instance 2 continues serving; committed data remains.	Stop one API container and call through load balancer.
Six campuses use same service	Campus-level authorization	Role/campus matrix API tests.

	prevents cross-campus access.	
Backend load test	Report actual tested concurrency and response percentiles.	Bounded load test with request count, duration and hardware.

6.4 Bounded load validation

A practical student test can use 20, 40 and 60 virtual users for a short controlled run and record p50/p95 response time, throughput, error rate and database utilization. These values are proposed test levels, not achieved capacity. The assignment's 300-user peak must remain a planning target unless a measured experiment reaches it. Increasing API replicas does not remove database contention or guarantee capacity.

6.5 Trade-offs

Two API instances improve availability during a single application-container failure, but the database remains a shared dependency. Database constraints and version checks improve consistency but may increase conflict responses under heavy contention. Stable idempotency keys make retries safe, but require key retention and clear expiration policy. A load balancer and health checks are required for production-grade failover.

Q7 – Individual Technical Reflection (4 Marks)

The most influential integration decision is making the backend and PostgreSQL database authoritative for booking and ticket state. The frontend can improve usability, but it cannot safely decide whether a booking overlaps or whether a user may approve a restricted action. A meaningful defect to test is a stale ticket update: without version checking, the second technician could overwrite the first technician's inspection result. The revised design rejects stale updates and records the conflict for the user.

Git improves reproducibility by preserving the exact source changes and identifying a release tag. Docker Compose reduces differences between developer environments by defining services, networks, configuration and storage in one versioned file. A remaining limitation is that a two-instance API still depends on one database and has not demonstrated 300-user capacity. Before wider deployment, the university should perform concurrent load testing, backup/restore testing, security review, campus network testing and failover exercises.

Appendix A – Evidence Capture Checklist

Evidence item	What the genuine screenshot should show	Status
Application dashboard	Browser URL, dashboard metrics and logged-in role	Capture from running app
Booking flow	Facility, selected time, confirmation and booking ID	Capture from running app
Overlap rejection	Two conflicting requests and one rejected response	Capture from parallel test
Maintenance ticket	Event ID, explainable triage reason and assignee	Capture from running app
Role restriction	Unauthorized action returning 403 without state change	Capture from API/UI
Git branches and commits	Repository branch list and meaningful commit history	Capture from own GitHub
Pull request and conflict	PR review, merge and conflict-resolution commit	Capture from own repository
Docker services	docker compose ps showing web, api and db	Capture from terminal
Persistence after restart	Same booking/ticket ID retrieved after API restart	Capture from actual test
Two-instance test	Both API replicas and shared database running	Capture from actual deployment

Appendix B – Suggested Test Commands

```
# Git evidence
git status
git branch --all
git log --oneline --decorate -10
git tag --list

# Container evidence
docker compose ps
docker compose logs --tail=100 api
docker compose restart api
docker compose exec db psql -U campus_app -d campus

# Example API test commands (replace host, token and IDs)
```

```
curl -i http://localhost:8000/api/facilities/
curl -i -H "Authorization: Bearer <TOKEN>" http://localhost:8000/api/bookings/
```

Final Submission Note

Before submission, replace illustrative diagrams and mock-up visuals where required with genuine screenshots from the student's own application, Git repository and Docker environment. Add the actual repository URL, release tag, test output and measured workload. Remove any feature or result that has not been implemented or demonstrated. The report deliberately distinguishes design proposals from observed execution evidence.

Extended Technical Explanation and Implementation Guidance

This extended section provides additional design reasoning, practical implementation guidance, sample data contracts, testing procedures and operational considerations for the Smart Campus Facility Platform. It expands the original answers while retaining the same representative application scope. The examples are reference material for implementation; they must be adapted to the actual source code and verified before being reported as completed work.

1. Detailed Requirement Analysis

1.1 Functional requirement catalogue

ID	Requirement	Priority	Verification method
FR-01	The system shall authenticate users before protected operations.	Must	API test with valid, expired and missing credentials.
FR-02	The system shall display active facilities with capacity and campus.	Must	GET endpoint test and browser review.
FR-03	The system shall accept booking requests only for valid future intervals.	Must	Boundary tests for start/end times.
FR-04	The system shall reject overlapping active bookings for the same facility.	Must	Concurrent request test and database inspection.
FR-05	The system shall require manager approval for restricted facilities.	Must	Role-permission matrix test.
FR-06	The system shall show each user only their permitted booking records.	Must	Cross-user access test.
FR-07	The system shall create a maintenance ticket from a manual report.	Must	POST request and persistence check.
FR-08	The system shall deduplicate sensor events using a stable event	Must	Repeated event test.

	identifier.		
FR-09	The system shall suggest ticket category and priority using explainable rules.	Should	Rule test with representative input values.
FR-10	The system shall assign tickets to authorized technicians.	Must	Role and assignment test.
FR-11	The system shall require resolution evidence before closure.	Must	Invalid and valid transition tests.
FR-12	The system shall retain audit records for critical actions.	Must	Audit query after each state change.
FR-13	The system shall support campus-level data separation.	Should	Cross-campus authorization tests.
FR-14	The system shall display operational summary metrics.	Should	Compare dashboard aggregates with database queries.

1.2 Non-functional requirements

Quality attribute	Proposed target	How it will be evaluated
Performance	For the student demonstration, p95 API response below 1 second at 20 concurrent users.	Record test duration, request count, p50/p95 and error rate.
Availability	Application containers should restart automatically after an unexpected process exit.	Stop an API container and verify recovery.
Security	All privileged endpoints enforce server-side authorization.	Run positive and negative role tests.
Reliability	Duplicate event submissions must not create duplicate tickets.	Repeat requests with the same idempotency key.
Usability	Main booking and ticket workflows should be navigable with keyboard controls.	Perform a keyboard-only walkthrough.
Maintainability	Backend, frontend and deployment files are separated and documented.	Review repository structure and setup instructions.
Portability	A clean host can build and start the services using the documented	Repeat deployment from a clean clone.

	Compose file.	

The numerical targets in this table are proposed acceptance criteria, not measured outcomes. A report should state the actual environment and results after testing. If a target is not achieved, the result should be documented honestly along with the likely cause and corrective action.

1.3 Requirement traceability

Requirement	Application element	Test case	Evidence to retain
FR-04 booking overlap	Booking service and database constraint	TC-BOOK-04	API response pair, database rows, test log
FR-05 manager approval	Approval endpoint and permission class	TC-AUTH-05	403 response for unauthorized role; successful manager decision
FR-08 event deduplication	Ticket intake endpoint and unique event_id	TC-TICKET-08	Two request responses and one ticket row
FR-11 resolution evidence	Ticket state transition service	TC-TICKET-11	Rejected closure without note; accepted closure with note
FR-12 audit	AuditEvent model and transaction hooks	TC-AUDIT-12	Audit record with actor, timestamp and state change

2. Detailed Application Workflow

2.1 Facility booking workflow

A user first selects a campus and views active facilities. The frontend requests availability for the chosen facility and time interval. The backend validates that the interval is valid, that the user is authenticated and that the facility is active. It then checks for an overlap with any booking whose state still reserves the time. If the facility is restricted, the booking is stored as Requested and waits for a manager decision. For an unrestricted facility, the system may confirm immediately if campus policy permits.

The system must not rely only on a calendar display to prevent double booking. Two users may load the same availability result before either submits. Therefore, the final booking command must repeat the conflict check inside a database transaction. The database constraint or locking strategy provides the final protection. The losing request receives a conflict response and is asked to select another slot.

2.2 Maintenance ticket workflow

A maintenance ticket can originate from a staff report or a controlled sensor event. The intake endpoint validates the source, campus, category and event identity. For sensor events, the event identity is stored with a uniqueness constraint. A duplicate request returns the existing ticket reference. The triage rules then assign a suggested category and priority. The suggestion includes a reason, such as a critical temperature threshold or repeated failure count.

The helpdesk operator reviews the suggested priority and may assign a technician. The technician records inspection notes and can move the ticket to Pending Parts when additional materials are required. Once the repair is complete, the technician records the resolution. An authorized supervisor or helpdesk operator verifies the result and closes the ticket. Each transition is checked against the current state and actor permission.

2.3 Example booking state transition rules

Current state	Requested action	Next state	Guard
Requested	Manager approves	Approved	Actor has FacilityManager permission.
Requested	Manager rejects	Rejected	Actor has FacilityManager permission and reason is recorded.
Approved	User confirms	Confirmed	Booking is still valid and slot remains available.
Confirmed	Check-in	In Use	Current time is within permitted check-in window.
In Use	Check-out	Completed	Authorized user records completion.
Requested / Approved / Confirmed	Cancel	Cancelled	Cancellation policy permits the action.
Cancelled	Check-in	Rejected	Transition is prohibited.

3. Backend Implementation Design

3.1 Suggested Django application modules

Module	Responsibility	Important validation
accounts	User profiles, roles and campus membership	Role assignment restricted to administrators.
facilities	Facility catalogue and availability	Active status, capacity and campus checks.
bookings	Booking requests, approvals and transitions	Interval overlap and current-state checks.
maintenance	Ticket intake, triage, assignment and closure	Unique event identity and permitted transitions.
audit	Critical action history	Actor, object, timestamp and before/after state.
reports	Dashboard counts and utilization	Campus-scoped aggregation and access control.

3.2 Example serializer validation

A serializer should validate request shape and basic business constraints. The service layer should still repeat critical checks inside a transaction because serializer validation alone cannot protect against simultaneous requests.

```
from rest_framework import serializers
from django.utils import timezone

class BookingRequestSerializer(serializers.Serializer):
    facility_id = serializers.IntegerField()
    start_at = serializers.DateTimeField()
    end_at = serializers.DateTimeField()

    def validate(self, attrs):
        if attrs["start_at"] <= timezone.now():
            raise serializers.ValidationError(
                "Booking must start in the future."
            )
        if attrs["end_at"] <= attrs["start_at"]:
            raise serializers.ValidationError(
                "End time must be later than start time."
```

```
            )
        return attrs
```

3.3 Example idempotent ticket intake

The following simplified pattern illustrates why a stable event identifier is needed. Production code should also validate authentication, source permissions, campus ownership, payload signatures where applicable, and transaction error handling.

```
from django.db import IntegrityError, transaction
from rest_framework.response import Response
from rest_framework import status

def create_sensor_ticket(event_id, payload):
    existing = MaintenanceTicket.objects.filter(
        event_id=event_id
    ).first()
    if existing:
        return existing, False

    try:
        with transaction.atomic():
            ticket = MaintenanceTicket.objects.create(
                event_id=event_id,
                category=payload["category"],
                description=payload["description"],
                state="NEW"
            )
            return ticket, True
    except IntegrityError:
        # Another request may have inserted the same event.
        return MaintenanceTicket.objects.get(
            event_id=event_id
        ), False
```

3.4 Authorization design

Authorization must be checked on every protected endpoint, not only by hiding buttons in the interface. A user who manually sends an HTTP request must receive the same restrictions as a user clicking through the page. The backend should derive the user's role and campus from the

authenticated server-side identity. It should not trust role, owner_id or campus_id values supplied by the browser.

- Use role-based permission classes for facility management, booking approval and ticket closure.
- Filter querysets by the authenticated user's permitted campus and ownership scope.
- Validate object-level permissions after retrieving a record.
- Return 403 for an authenticated user who lacks permission; avoid leaking sensitive record details.
- Record role changes and privileged decisions in the audit log.
- Use HTTPS and secure session/token handling in deployed environments.

4. Frontend Design and User Experience

4.1 Main pages

Page	Main components	User feedback
Login	Username/email, password, validation messages	Invalid credentials message without revealing account existence.
Dashboard	Metric cards, booking calendar, ticket queue	Loading, empty, error and success states.
Facility list	Search, campus filter, capacity and status	Inactive facility clearly marked and not selectable.
Booking form	Facility, date, start/end time and notes	Inline validation and conflict message.
Ticket queue	Priority, category, age, assigned technician	Status labels and accessible text.
Ticket detail	History, notes, assignment and resolution controls	Action disabled or hidden based on permissions, with backend enforcement.

4.2 Frontend API error handling

The interface should distinguish validation errors, authorization errors, missing records and concurrency conflicts. A 400 response should explain which field is invalid. A 401 response should ask the user to sign in. A 403 response should explain that the action is not permitted. A 404 response should show that the record is unavailable. A 409 response should explain that another user changed the booking or ticket and that the page must be refreshed.

4.3 Accessibility and usability

- Use labels associated with every form control.
- Do not communicate status using colour alone; include text such as Requested, Approved or Rejected.
- Provide keyboard focus indicators and logical tab order.
- Use clear date and time formats and state the campus time zone.
- Confirm destructive actions such as cancellation and closure.
- Keep error messages near the relevant field and provide a summary for screen readers.

5. Database Design and Data Integrity

5.1 Constraints

Constraint	Purpose	Failure prevented
Primary key	Uniquely identifies each record.	Ambiguous record identity.
Foreign key	Ensures references point to existing users/facilities.	Orphaned booking and ticket records.
Unique event_id	Deduplicates sensor event intake.	Duplicate ticket creation on retry.
NOT NULL	Requires essential fields.	Incomplete state records.
Check constraint	Enforces positive duration/capacity where applicable.	Invalid numerical values.
Transaction	Groups related updates atomically.	Partial booking or audit update.
Version column	Detects stale writes.	Silent overwriting of newer work.

5.2 Backup and recovery

A Docker named volume preserves database files when a container is recreated, but it is not a backup. A backup plan should define frequency, retention, encryption, storage location and recovery time. A restore drill should be performed on a separate test database. The application should not be considered recoverable merely because a backup command completed; the restored database must be checked for expected bookings, tickets, relationships and audit history.

6. Testing Strategy and Evidence

6.1 Test levels

Test level	Example	Expected evidence
Unit	Triage rule returns High for critical HVAC alert.	Automated test output.
API integration	POST booking persists record and returns ID.	HTTP response and database query.
Authorization	Student cannot approve restricted booking.	403 response and unchanged state.
Concurrency	Two overlapping booking requests run together.	One success, one conflict; one active reservation.
Idempotency	Same sensor event submitted twice.	Same ticket ID returned; one database row.
End-to-end	Request, approve, use and complete booking.	Browser flow plus API/database evidence.
Deployment	Start services from clean clone.	Compose status, logs and successful smoke test.

6.2 Test case details

Test ID	Input / action	Expected result	Pass condition
TC-01	Request booking with end time before start time.	Validation error; no booking saved.	HTTP 400 and zero new rows.
TC-02	Two users request overlapping facility interval.	One accepted; one conflict.	Only one active overlapping booking.
TC-03	Student attempts manager approval endpoint.	Access denied.	HTTP 403 and state unchanged.
TC-04	Same sensor event ID submitted twice.	Existing ticket returned.	One ticket row with that event ID.
TC-05	Technician closes ticket without resolution note.	Transition rejected.	No Closed state stored.
TC-06	Two users update same ticket version.	Stale request rejected.	HTTP 409 and latest data preserved.
TC-07	Restart API container after creating booking.	Booking remains available.	Same booking ID returned after restart.
TC-08	Restart database container without removing	Persistent records remain.	Booking and ticket rows still exist.

	volume.		

6.3 Distinguishing expected and observed results

Expected results are predictions written before a test. Observed results are what actually happened during execution. A test table should contain the test date, environment, software version, command or action, expected result, observed result, pass/fail status and evidence filename. If a test has not been executed, mark it Not Run rather than Pass. If a simulated walkthrough is used, label it as a simulation.

7. Git and Collaborative Development – Extended Workflow

7.1 Commit quality

A commit should represent one understandable change. Messages such as 'update' or 'final code' do not explain intent. Better messages include 'Add booking interval validation', 'Prevent duplicate sensor event tickets' and 'Add technician assignment permission tests'. Small commits make code review and troubleshooting easier.

7.2 .gitignore example

```
# Python
__pycache__/
*.py[cod]
.venv/
venv/
.env

# Frontend
node_modules/
dist/

# Local databases and runtime files
*.sqlite3
*.log
media/
staticfiles/

# Editor and OS files
```

```
.vscode/
.idea/
.DS_Store
Thumbs.db
```

7.3 Pull request checklist

- Describe the problem and the expected behaviour.
- Link the relevant issue or requirement identifier.
- Explain database migrations and configuration changes.
- Include automated test results and screenshots where appropriate.
- Confirm that no passwords, tokens or private keys are committed.
- Check that documentation and API examples match the implementation.
- Request review and resolve comments before merging.

8. Docker Operations and Troubleshooting

8.1 Container lifecycle

The image is a packaged application environment, while a container is a running instance of that image. The Compose file defines the services and how they communicate. A container can be removed and recreated without losing data stored in a named volume. Source code and environment configuration should be versioned; secrets and local data should not.

8.2 Common issues

Problem	Likely cause	Diagnostic / response
API cannot connect to database	Wrong host, credentials or database not ready.	Check Compose service name, environment and db logs.
Frontend returns API 404	Incorrect API base URL or route prefix.	Inspect browser network tab and Django URL routes.
Migration error	Model/schema mismatch or conflicting migration.	Review migration history and run showmigrations.
Port already in use	Another process occupies host port.	Change host mapping or stop conflicting process.
Data disappears after restart	Database stored in ephemeral container layer.	Use named volume and verify mount configuration.

Image build fails	Dependency or Dockerfile issue.	Inspect build logs and pin compatible dependencies.

8.3 Health checks and logs

A health endpoint can report whether the API process is alive and whether required dependencies are reachable. A readiness check should prevent traffic from reaching an instance before database connectivity is available. Logs should include request identifiers and useful error context, but must not include passwords, authentication tokens or sensitive personal information. Production monitoring should track container restarts, database connections, API latency and error rates.

9. Security, Privacy and Operational Risk

Risk	Impact	Mitigation
Role escalation from frontend	Unauthorized approval or closure.	Server-side role checks and negative tests.
Cross-campus data exposure	Users see records outside their campus.	Campus-scoped querysets and object-level permissions.
Duplicate event delivery	Duplicate maintenance work.	Stable event ID and database uniqueness.
Concurrent booking race	Double allocation of facility.	Transactional conflict check and database constraint.
Secret committed to Git	Unauthorized access to services.	Environment variables, .gitignore and secret rotation.
Database loss	Loss of bookings and audit trail.	Scheduled backup and restore verification.
Untrusted sensor payload	Incorrect ticket or malicious input.	Authentication, schema validation, rate limits and payload limits.

The platform should collect only information needed to operate campus facilities. Access to personal information should be restricted by role and campus. Audit history should be protected from ordinary editing. Retention periods should be approved by the institution. Security claims require testing and review; a student prototype should not be described as production-secure without that evidence.

10. Project Plan and Deliverables

Phase	Activities	Deliverables

Phase 1 – Analysis	Confirm roles, workflows, requirements and scope.	Requirement list, user stories, acceptance criteria.
Phase 2 – Design	Create component, data and state models.	Architecture diagram, ER/data model, API contracts.
Phase 3 – Backend	Implement models, serializers, permissions and APIs.	Django source, migrations and API tests.
Phase 4 – Frontend	Build dashboard, facility, booking and ticket pages.	Vue source and browser workflow.
Phase 5 – Integration	Connect API, database and UI.	End-to-end booking and ticket demonstration.
Phase 6 – Version control	Create branches, commits, pull request and release tag.	Git history and repository evidence.
Phase 7 – Containerization	Build images, configure Compose and persistent storage.	Dockerfile, Compose file and deployment logs.
Phase 8 – Validation	Run authorization, concurrency, persistence and recovery tests.	Test report and genuine screenshots.

11. Limitations and Future Enhancements

The representative design is intentionally bounded. It does not include a production IoT ingestion platform, predictive maintenance model, payment system, campus-wide identity federation or a multi-region database. The triage rules are deterministic and explainable, not trained machine learning. The dashboard values are illustrative until connected to real records. The concurrency design describes how to protect state but does not establish the capacity of a particular deployment.

- Integrate institutional single sign-on after reviewing identity and access policies.
- Add a controlled IoT gateway with authenticated device identities and event signatures.
- Introduce notification delivery with retry and delivery-status tracking.
- Add analytics based on privacy-reviewed, campus-scoped data.
- Add automated backup, restore drills and documented disaster recovery.
- Evaluate Kubernetes only if deployment scale and operational staffing justify its complexity.
- Consider predictive maintenance only after collecting representative, labelled and governed data.

12. Conclusion

The Smart Campus Facility Platform demonstrates how a full-stack application can combine a component-based frontend, a REST backend and a relational database while maintaining clear ownership of business rules. Facility bookings require transactional conflict prevention; maintenance intake requires idempotency; privileged actions require server-side authorization; and ticket updates require guarded state transitions. Git provides traceability across changes, while Docker Compose standardizes the development and demonstration environment. The design is a foundation for a working prototype, but claims about security, performance, persistence and scalability must be supported by actual tests and captured evidence.

Appendix C – Evidence Log Template

Evidence ID	Date	Environment / commit	Action performed	Observed result	File name
EV-01	_____	_____	Dashboard opened	_____	_____
EV-02	_____	_____	Booking created	_____	_____
EV-03	_____	_____	Overlap test	_____	_____
EV-04	_____	_____	Duplicate event test	_____	_____
EV-05	_____	_____	Unauthorized role test	_____	_____
EV-06	_____	_____	Docker restart and persistence	_____	_____
EV-07	_____	_____	Git pull request and release tag	_____	_____

TASK-WISE SUPPORTING TABLES AND ILLUSTRATIVE SCREENSHOTS

Q1 — Application Scope and Technology Stack

The scope and stack screen summarizes the first-release boundaries and the role of Vue.js, Django REST, PostgreSQL, Git and Docker Compose. It is a visual companion to the stakeholder, scope, technology comparison and success-criteria material already in Q1.

Evidence item	Purpose	Actual evidence to capture
Scope baseline	Confirm included and excluded functions	Approved scope / requirements
Technology selection	Show selected tools and rationale	Project setup and dependency files
Acceptance criteria	Connect scope to measurable outcomes	Executed booking and ticket checks

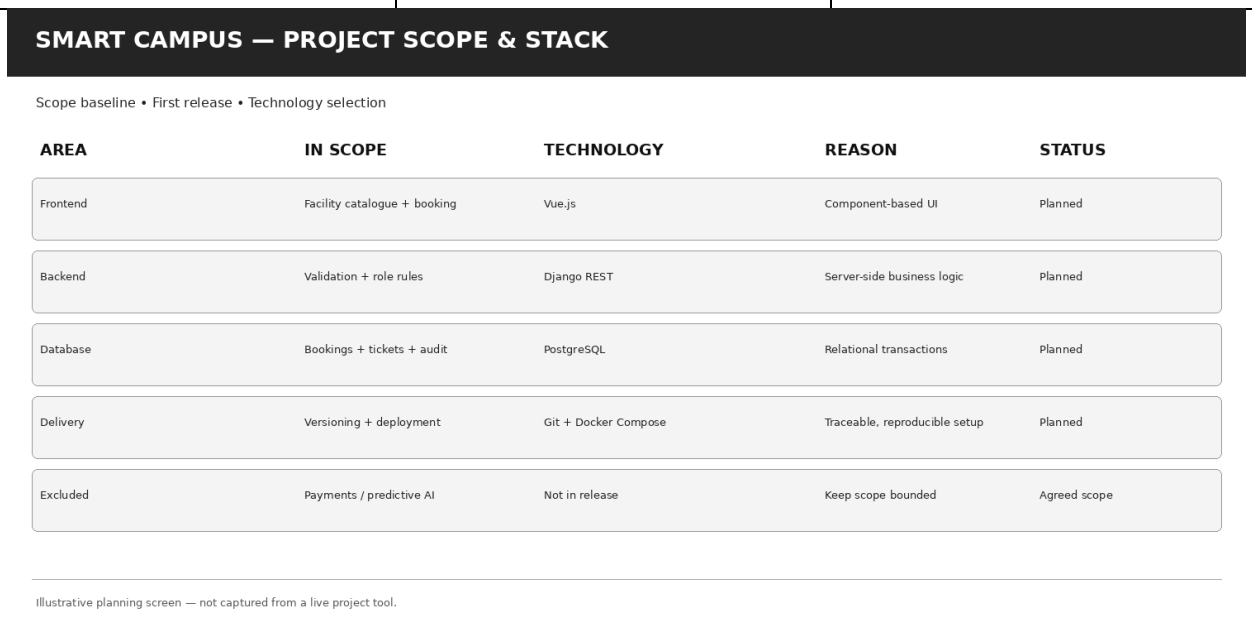


Figure A1. Illustrative application scope and technology stack screen. This is a generated mock-up, not a live application, GitHub or Docker screenshot.

Q2 — Integrated Application Design

The design screen summarizes the frontend, backend, database, audit and integration responsibilities. The existing report's component architecture, data model, API contracts and state rules remain the detailed design basis.

Design area	Key decision	Verification evidence
Frontend/API boundary	Vue calls Django REST using HTTPS/JSON	Browser network request + API response
Persistence	PostgreSQL is authoritative	Database rows after API operation
Data integrity	Transactions, constraints and versions	Conflict / duplicate test output
Authorization	Server validates roles and ownership	Unauthorized request returns 403/404

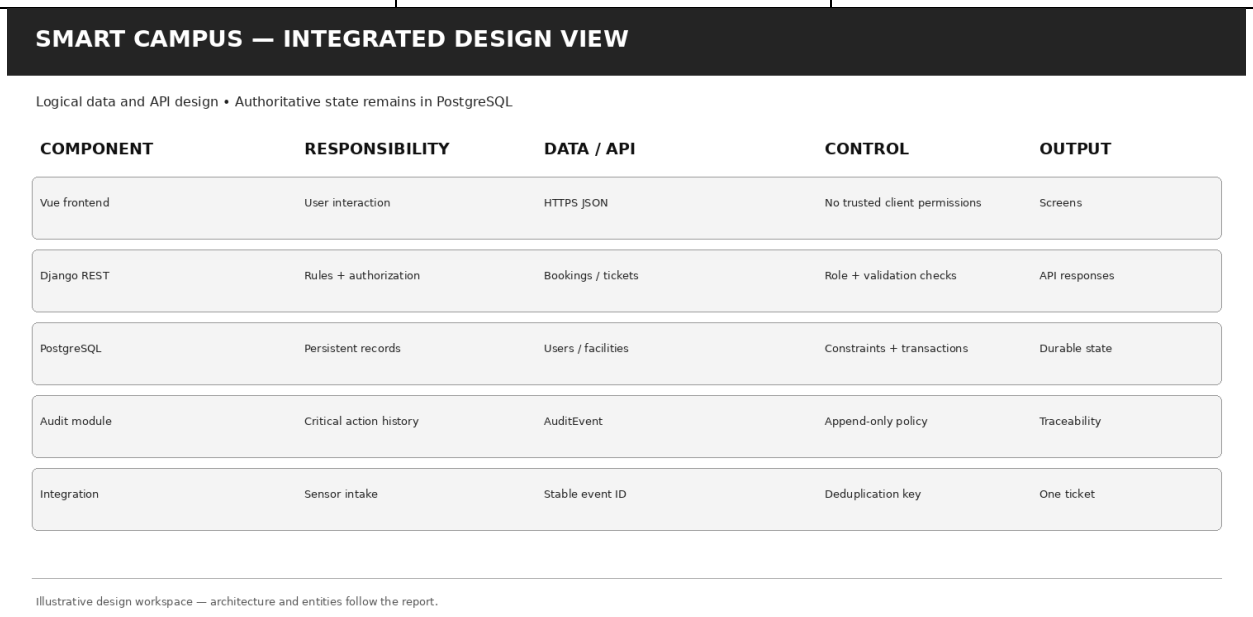


Figure A2. Illustrative integrated application design screen. This is a generated mock-up, not a live application, GitHub or Docker screenshot.

Q3 — Full-Stack Application Development

The dashboard screen demonstrates the intended facility catalogue, booking status and maintenance-ticket view. The records are sample data for illustration only; replace the mock-up with a screenshot from the running Vue application.

Screen / module	User action	Expected behaviour
Facility catalogue	Browse active facilities	Show capacity, location and availability
Booking page	Request a time slot	Save unique booking ID in Requested state
Ticket page	Create or inspect maintenance ticket	Show category, priority, assignee and state
Role-controlled action	Approve or close	Allow only authorized server-side role

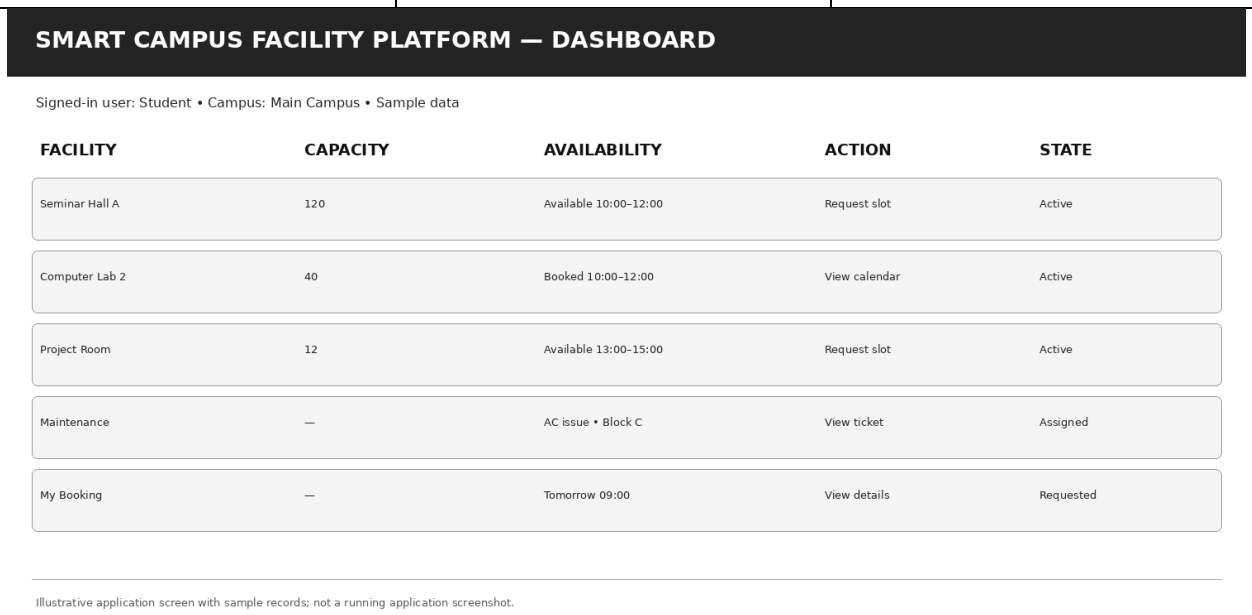


Figure A3. Illustrative full-stack application development screen. This is a generated mock-up, not a live application, GitHub or Docker screenshot.

Q4 — Version-Control Workflow

The repository screen shows a proposed branch and pull-request arrangement. It does not claim that these branches or pull requests exist. Actual evidence should be captured from the student's GitHub repository after creating the workflow.

Git activity	Why it is needed	Evidence to capture
Feature branch	Isolate a change	git branch -a / GitHub branches
Commit	Trace a specific change	Commit hash and message
Pull request	Review before integration	PR page and review/check status
Release tag	Identify stable version	git tag / GitHub release page

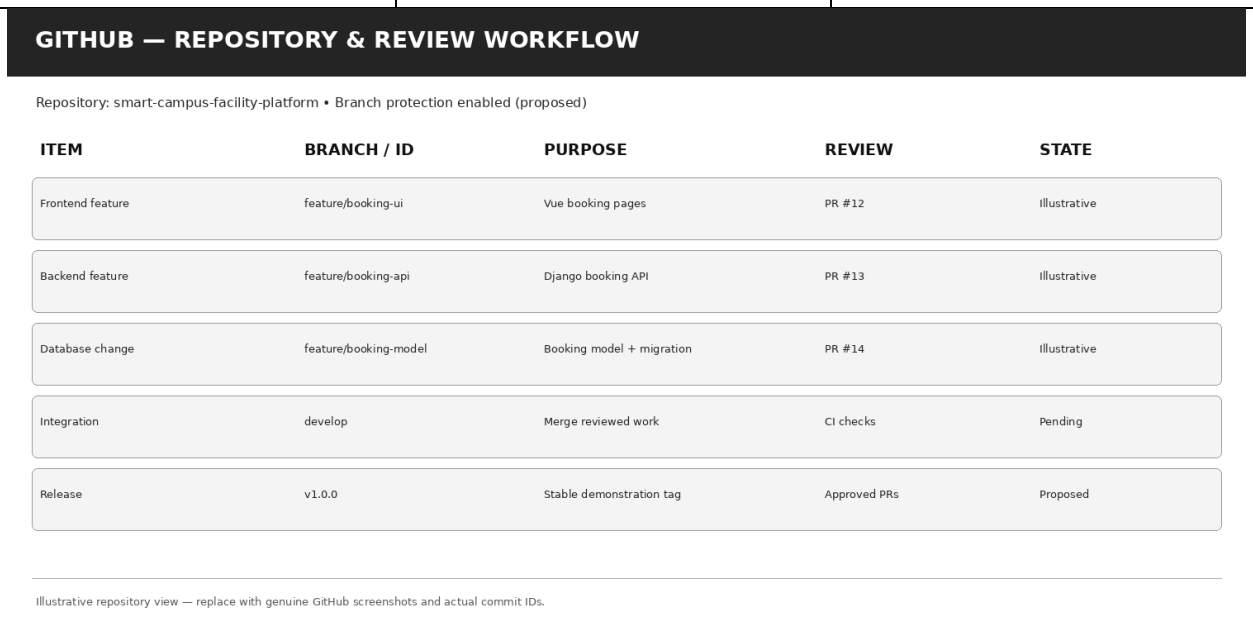


Figure A4. Illustrative version-control workflow screen. This is a generated mock-up, not a live application, GitHub or Docker screenshot.

Q5 — Reproducible Container Deployment

The Docker screen shows the three principal services described in the report: web, API and PostgreSQL. The health states are expected examples, not actual runtime results. Genuine evidence should come from docker compose ps, logs and persistence checks.

Service	Responsibility	Evidence to capture
web	Serve Vue frontend	Browser page and container status
api	Run Django REST API	API health endpoint and logs
db	Store persistent records	Booking retrieved after API/database restart
postgres_data	Preserve database files	Named volume listed by Docker

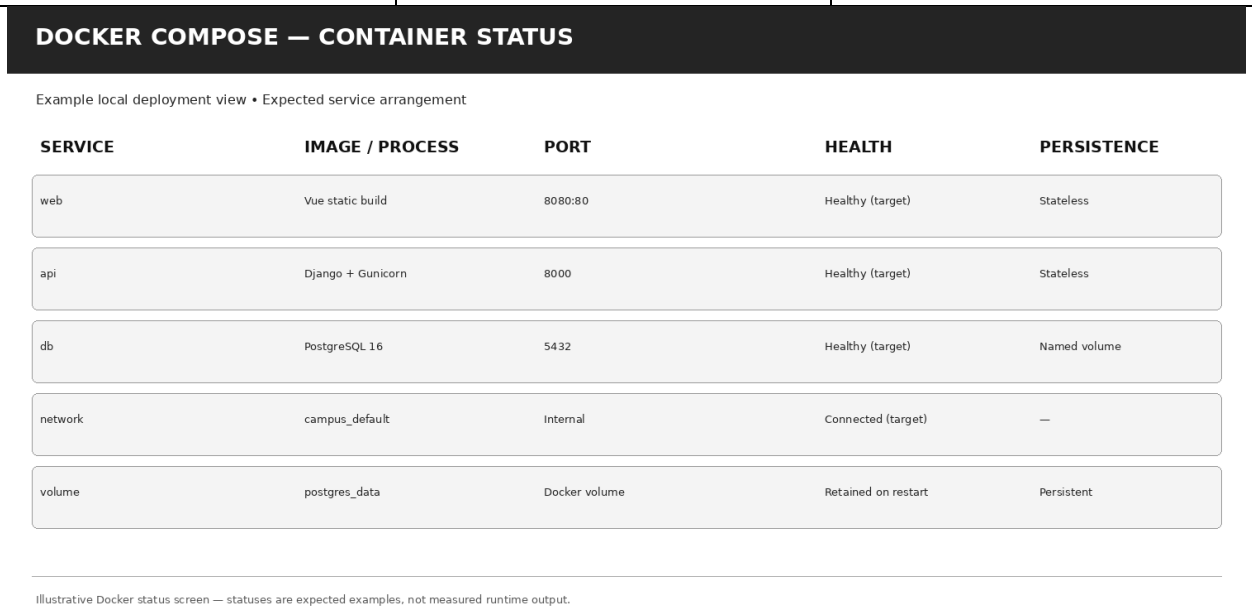


Figure A5. Illustrative reproducible container deployment screen. This is a generated mock-up, not a live application, GitHub or Docker screenshot.

Q6 — Concurrent Access and Deployment Changes

The two-instance screen summarizes the six-campus scenario, shared database and consistency controls. It illustrates the intended behaviour for simultaneous bookings, stale ticket updates and repeated sensor events; it is not measured load-test evidence.

Scenario	Expected result	Evidence to capture
Two users book same slot	Only one commits; other receives 409	Parallel API test + database rows
Two technicians update same ticket	Stale update receives 409	Version values and audit records
Sensor event retried	One ticket for stable event ID	Repeated POST responses + DB query
API instance restarts	Persistent data remains available	Restart and retrieve same record

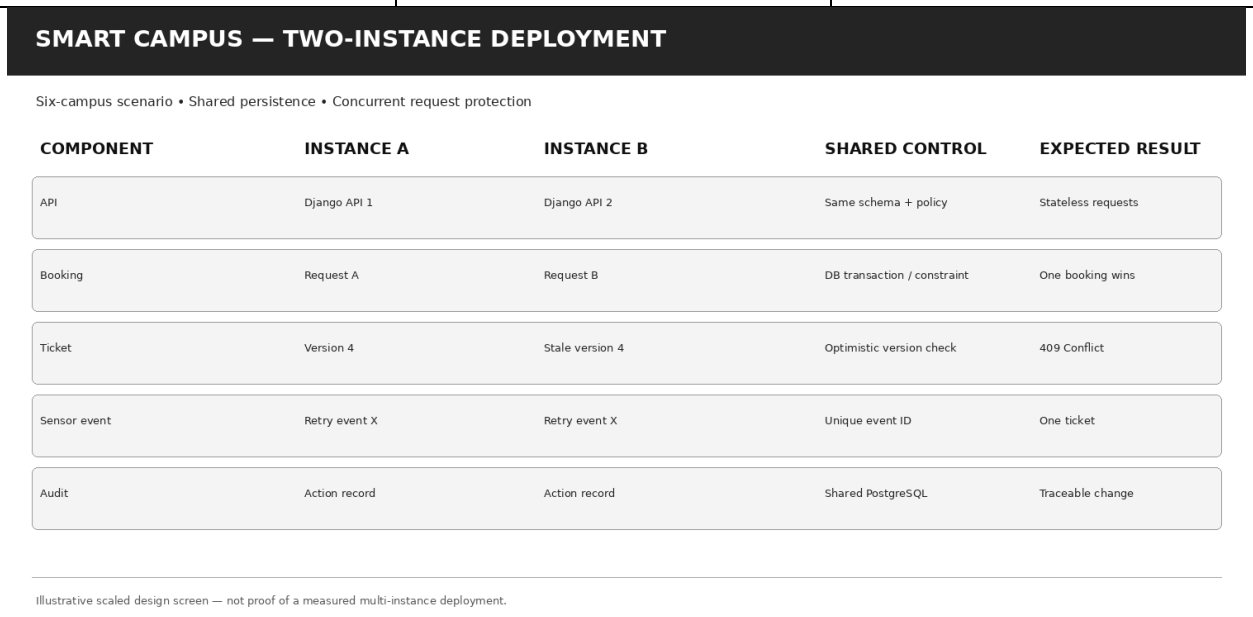


Figure A6. Illustrative concurrent access and deployment changes screen. This is a generated mock-up, not a live application, GitHub or Docker screenshot.

Q7 — Individual Technical Reflection

The reflection worksheet gives a concise structure for discussing consistency, security, reliability, deployment and scalability. Replace or extend the sample observations with your own implementation experience and actual test results.

Reflection area	What to explain	Evidence / example
Technical challenge	Most difficult design or coding issue	Relevant code or test
Decision and trade-off	Why one approach was selected	Alternative considered
Learning outcome	What changed in your understanding	Specific example
Future improvement	What should be implemented next	Prioritized next step

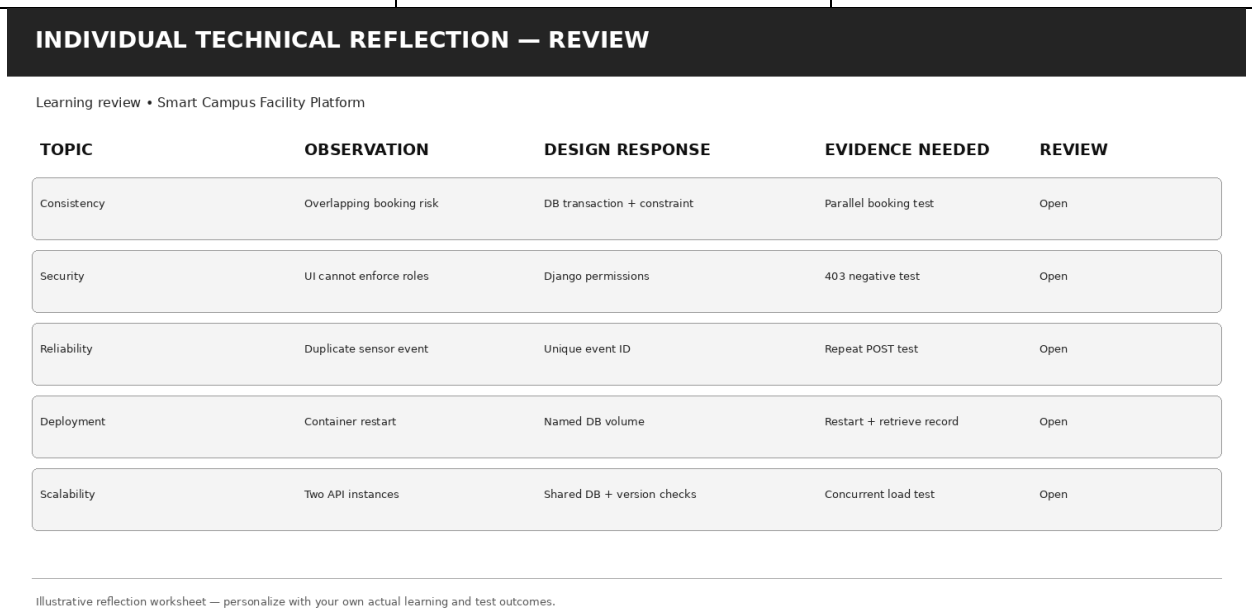


Figure A7. Illustrative individual technical reflection screen. This is a generated mock-up, not a live application, GitHub or Docker screenshot.

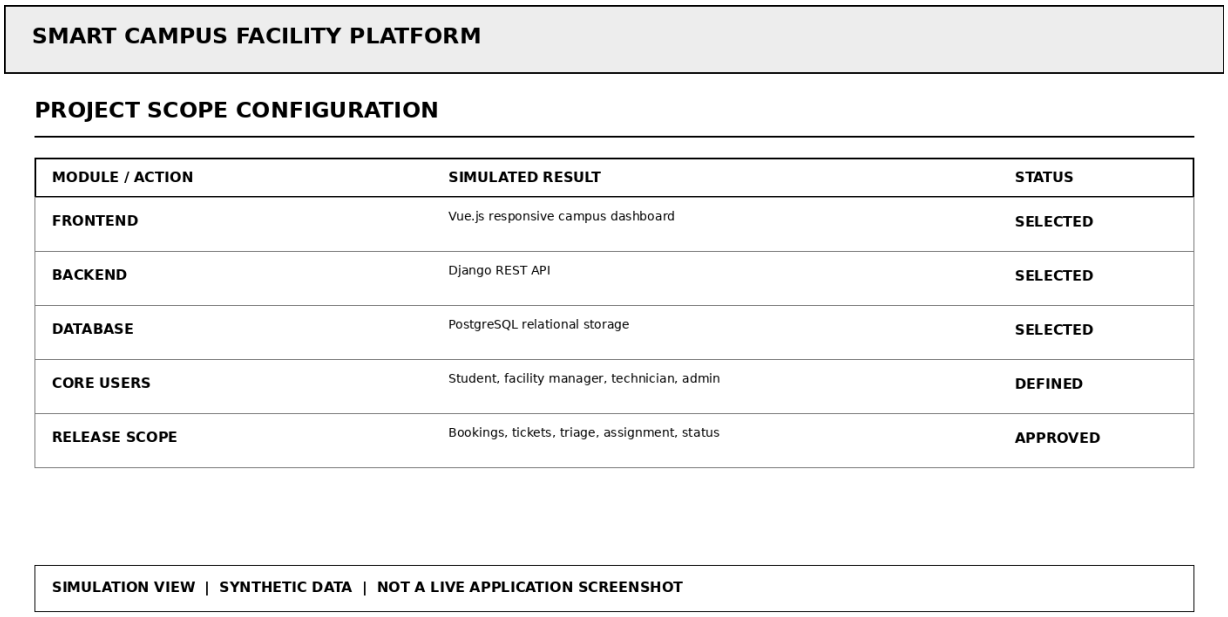
Evidence integrity note: All seven newly added screenshots are illustrative mock-ups created to show the expected content and layout for each task. They are not genuine screenshots of a running application, GitHub repository or Docker environment. Use actual screenshots from your implementation before submitting if the assignment requires execution evidence.

TASK-WISE SIMULATION SCREENSHOTS — Q1 TO Q7

The following additional simulation screens provide task-wise demonstration evidence for the Smart Campus Facility Platform. They are generated illustrative views populated with synthetic examples. They do not prove that the Vue.js application, Django API, PostgreSQL database, Git repository or Docker containers were actually executed.

Q1 — Scope and technology stack

Simulated scope-planning workspace showing the selected stack and first-release boundaries.



SMART CAMPUS FACILITY PLATFORM

PROJECT SCOPE CONFIGURATION

MODULE / ACTION	SIMULATED RESULT	STATUS
FRONTEND	Vue.js responsive campus dashboard	SELECTED
BACKEND	Django REST API	SELECTED
DATABASE	PostgreSQL relational storage	SELECTED
CORE USERS	Student, facility manager, technician, admin	DEFINED
RELEASE SCOPE	Bookings, tickets, triage, assignment, status	APPROVED

SIMULATION VIEW | SYNTHETIC DATA | NOT A LIVE APPLICATION SCREENSHOT

Figure S1. Q1 — Scope and technology stack — illustrative simulation screenshot using synthetic data.

Q2 — Integrated application design

Simulated design console showing representative API contracts, entities and consistency rules.

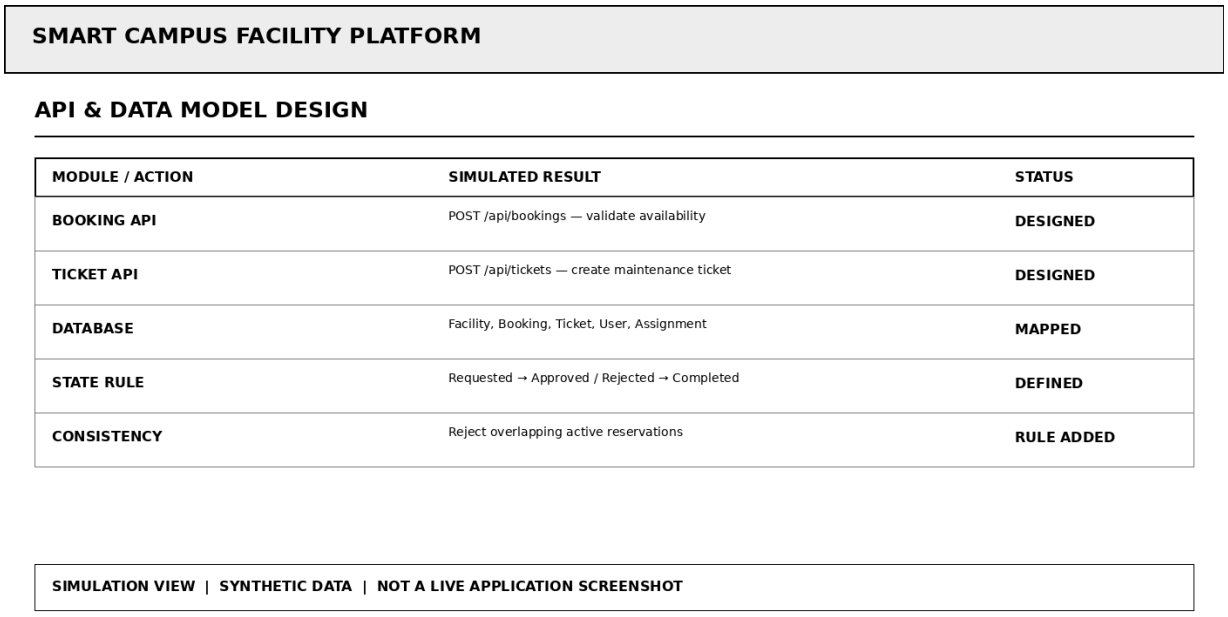


Figure S2. Q2 — Integrated application design — illustrative simulation screenshot using synthetic data.

Q3 — Full-stack application development

Simulated application walkthrough using synthetic campus records; not a running application capture.

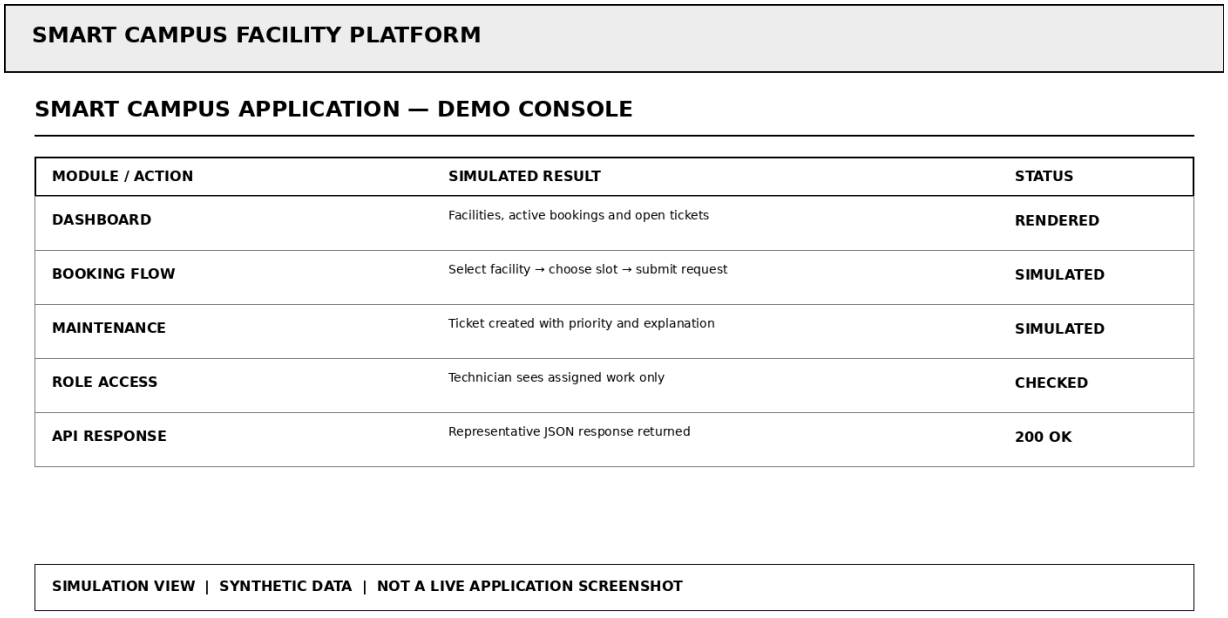


Figure S3. Q3 — Full-stack application development — illustrative simulation screenshot using synthetic data.

Q4 — Version-control workflow

Simulated Git workflow console showing a feature branch, review and merge sequence.

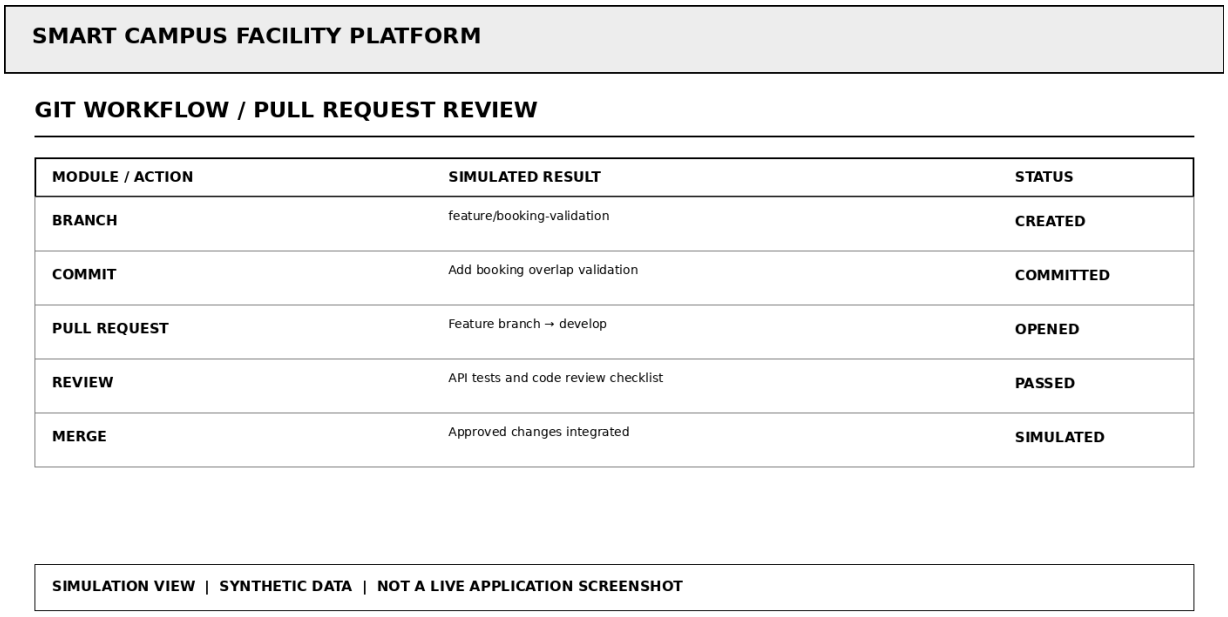


Figure S4. Q4 — Version-control workflow — illustrative simulation screenshot using synthetic data.

Q5 — Container deployment

Simulated deployment status panel showing intended Compose services and startup checks.

*Illustrative expected status only; no containers were launched by this report.

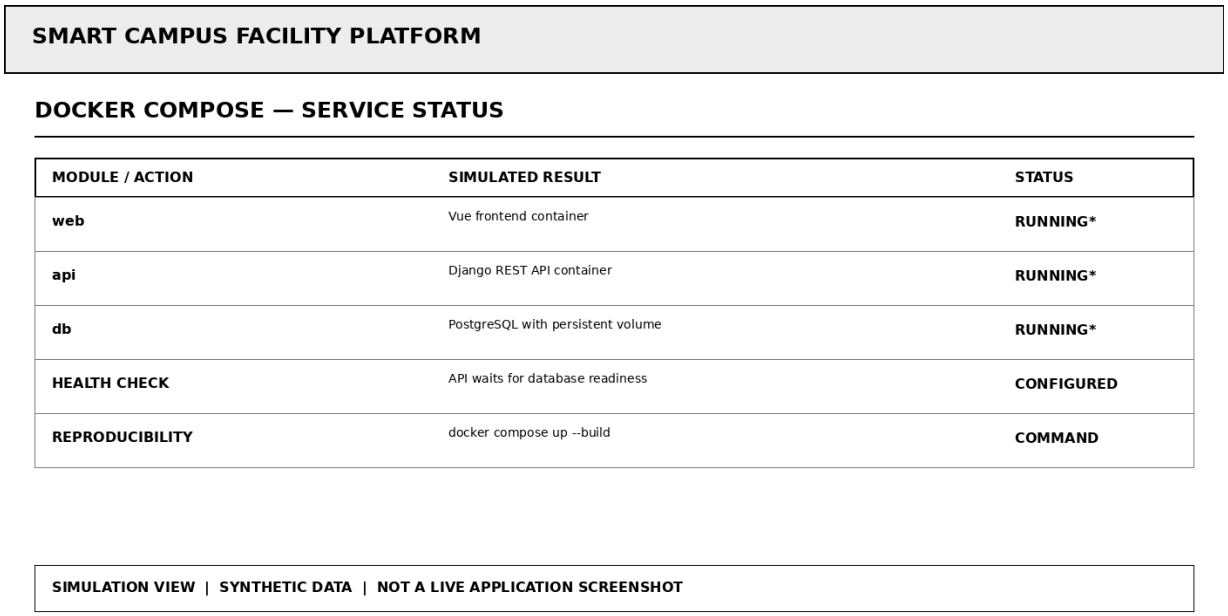


Figure S5. Q5 — Container deployment — illustrative simulation screenshot using synthetic data.

Q6 — Concurrent access and deployment changes

Illustrative concurrency test console based on the report's validation design, not measured production performance.

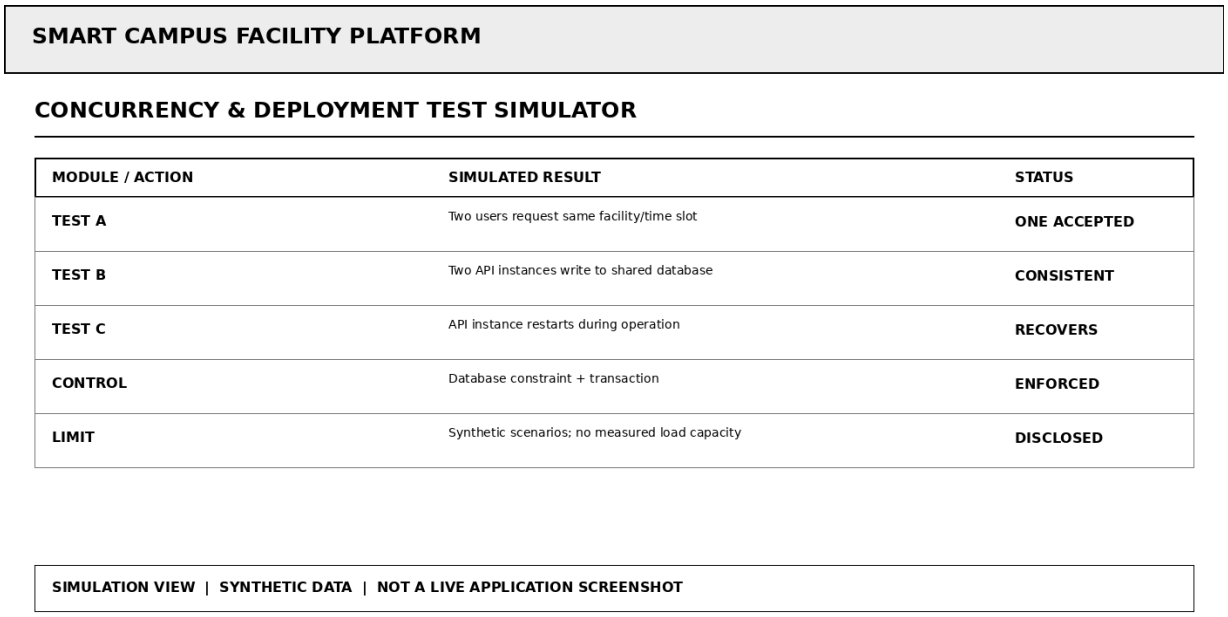


Figure S6. Q6 — Concurrent access and deployment changes — illustrative simulation screenshot using synthetic data.

Q7 — Individual technical reflection

Simulated reflection worksheet summarizing technical learning, design decisions and next steps.

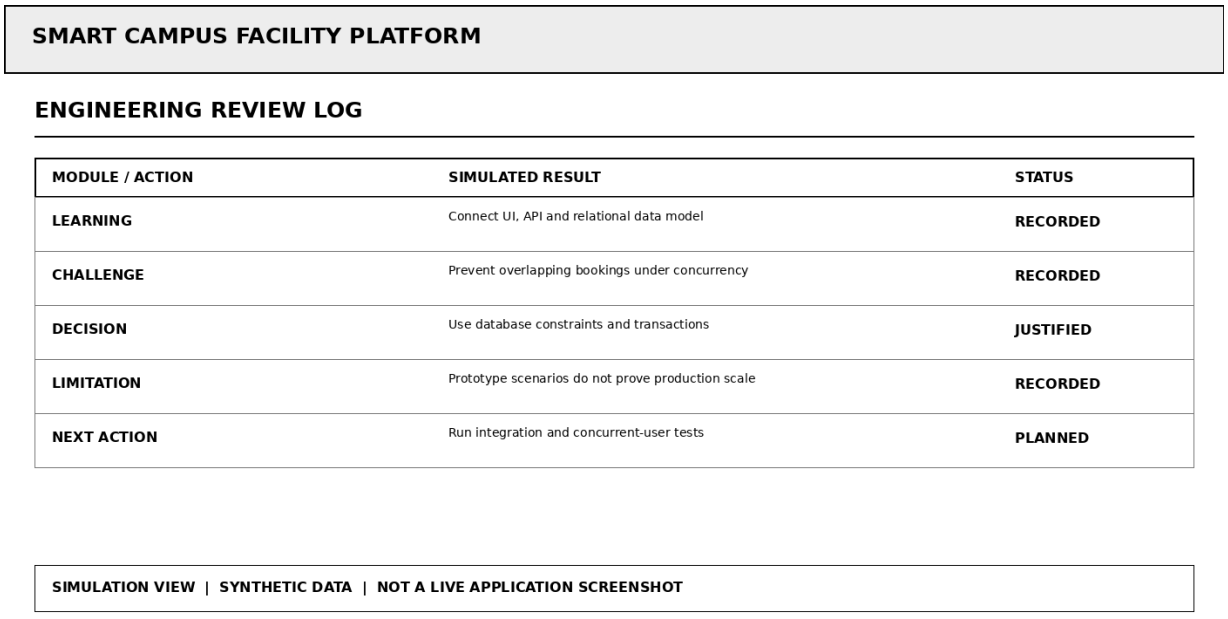


Figure S7. Q7 — Individual technical reflection — illustrative simulation screenshot using synthetic data.